

NUTRIALIANZA S.A.

Sistema de Monitoreo Inteligente con Docker e IA

Resumen académico

ESCENARIO 2

Saturación de conexiones MySQL

Actividad	Tarea 4 - Emulación de Saturación en Producción
Resultado general	Escenario completado con detección automática y alerta por Telegram
Fecha de prueba	15 de agosto de 2026
Integrantes	Julio Gutiérrez Orozco María José Miranda López

Objetivo: comprobar el comportamiento de MySQL ante un exceso de conexiones concurrentes y validar la cadena de detección Prometheus → N8N → Groq → Telegram.

1. Objetivo del escenario

El Escenario 2 tuvo como propósito provocar una saturación controlada del servidor MySQL mediante **mysqlslap**, superar el límite de conexiones concurrentes configurado en el servicio y comprobar que el sistema de monitoreo fuera capaz de reconocer el incidente, generar un análisis técnico con inteligencia artificial y enviar una alerta automática al equipo de TI por Telegram.

La prueba se diseñó sobre el límite real del entorno: **max_connections = 151**. No se redujo este valor para facilitar el resultado; la saturación se produjo al intentar abrir más conexiones que las permitidas por la configuración original del servidor.

2. Parámetros y resultados principales

Indicador	Valor observado	Interpretación
Límite de MySQL	151 conexiones	Capacidad máxima configurada antes de rechazar conexiones normales.
Concurrencia solicitada	220 clientes	La carga superó el límite en 69 clientes concurrentes.
Consultas ejecutadas	220	Cada cliente ejecutó una consulta de prueba.
Consulta utilizada	SELECT SLEEP(10)	Mantuvo las sesiones ocupadas el tiempo suficiente para generar contención.
Duración del benchmark	16.492 s	Tiempo total reportado por mysqlslap para la ejecución.
Error observado	1040 Too many connections	MySQL rechazó nuevas conexiones al alcanzar el límite.
Connection_errors_max_connections	897	Contador acumulado de intentos rechazados por límite de conexiones.
Threads_connected al finalizar	1	El servicio recuperó su estado normal una vez terminada la carga.
Alerta automática	Recibida en Telegram	La cadena Prometheus → N8N → Groq → Telegram funcionó correctamente.

3. Preparación del monitoreo

Antes de ejecutar la carga se validó la ruta completa de observabilidad de MySQL. El contenedor mysqld-exporter quedó disponible en el puerto 9104 y Prometheus incorporó un job específico para consultar sus métricas cada 15 segundos. La métrica mysql_up devolvió 1, lo que confirmó que el exporter podía comunicarse con el servidor.

También se verificaron dos indicadores clave: `mysql_global_status_threads_connected`, que reflejaba aproximadamente 2 conexiones en condiciones normales, y `mysql_global_status_connection_errors_total{error="max_connections"}`, cuyo valor inicial era 0. Esta línea base permitió distinguir con claridad el cambio producido por la prueba.

```

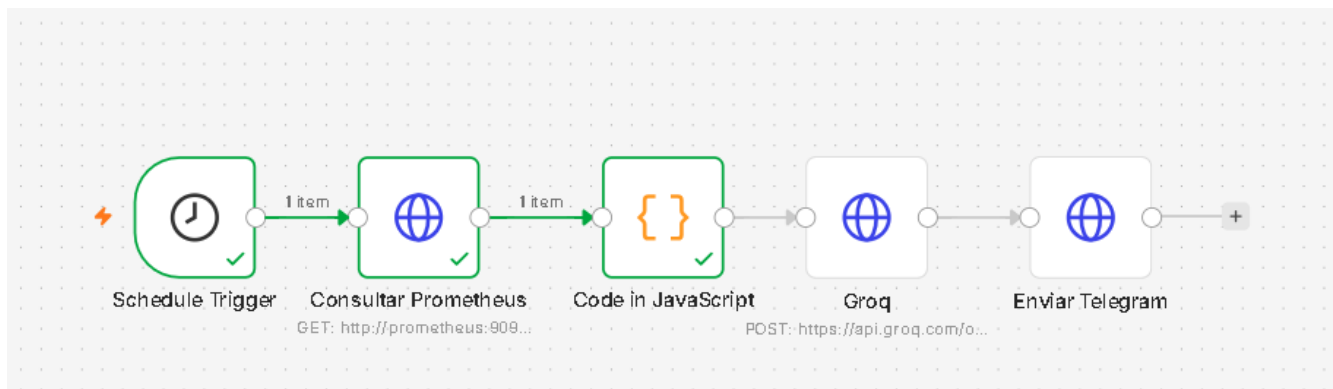
julio@DESKTOP-JN2R04E:~/nutrialianza-monitorer$ curl -s http://localhost:9104/metrics | grep "^mysql_up "
mysql_up 1
julio@DESKTOP-JN2R04E:~/nutrialianza-monitorer$ curl -s http://localhost:9104/metrics | grep -Ei "threads_connected|connection_
max_connections|max_connections"
mysql_global_status_connection_errors_total{error="max_connections"} 0
# HELP mysql_global_status_threads_connected Generic metric from SHOW GLOBAL STATUS.
# TYPE mysql_global_status_threads_connected untyped
mysql_global_status_threads_connected 2
# HELP mysql_global_variables_max_connections Generic gauge metric from SHOW GLOBAL VARIABLES.
# TYPE mysql_global_variables_max_connections gauge
mysql_global_variables_max_connections 151
# HELP mysql_global_variables_mysqlx_max_connections Generic gauge metric from SHOW GLOBAL VARIABLES.
# TYPE mysql_global_variables_mysqlx_max_connections gauge
mysql_global_variables_mysqlx_max_connections 100
julio@DESKTOP-JN2R04E:~/nutrialianza-monitorer$

```

Para N8N se creó una copia independiente del workflow utilizado en el Escenario 1. La consulta de Prometheus se cambió por:

```
increase(mysql_global_status_connection_errors_total{error="max_connections"}[1m])
```

La función `increase()` permite medir cuántos rechazos nuevos se produjeron durante la última ventana de un minuto. El nodo JavaScript no enviaba información a Groq mientras el valor fuera 0. Si el resultado superaba 0, el workflow construía el contexto del incidente, solicitaba el análisis a Groq y enviaba el texto resultante a Telegram. El Schedule Trigger quedó configurado cada 10 segundos para reducir el riesgo de perder un evento corto.



4. Preparación de mysqlslap

La imagen de MySQL utilizada en Docker no incluía la utilidad `mysqlslap`. La comprobación inicial dentro del contenedor devolvió “`mysqlslap: command not found`”. WSL tampoco la tenía instalada, por lo que se agregó el paquete `mysql-client-core-8.0`. Después de la instalación, el comando `mysqlslap --version` confirmó la versión 8.0.46 para Linux x86_64.

Antes de aplicar carga se validó la conexión desde WSL hacia MySQL en 127.0.0.1:3306 mediante un `SELECT 1`. La respuesta correcta confirmó que las credenciales, el puerto y la base de datos eran accesibles desde el cliente utilizado para la prueba.

```

julio@DESKTOP-JN2R04E:~/nutrialianza-monitoreo$ source .env
julio@DESKTOP-JN2R04E:~/nutrialianza-monitoreo$ mysql -h 127.0.0.1 -P 3306 \
-u"$M> -u"$MYSQL_USER" \
> -p"$MYSQL_PASSWORD" \
MYSQL> "$MYSQL_DATABASE" \
> -e "SELECT 1;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+
| 1 |
+----+
| 1 |
+----+

```

5. Ejecución de la saturación

La prueba se ejecutó con 220 clientes concurrentes frente a un límite de 151 conexiones. Esto generó una diferencia de 69 clientes por encima de la capacidad configurada. Cada cliente ejecutó `SELECT SLEEP(10)`, con una iteración y una consulta por cliente. El objetivo de `SLEEP(10)` fue mantener abiertas las sesiones durante varios segundos para que la concurrencia coincidiera en el tiempo y la saturación fuera observable.

```

julio@DESKTOP-JN2R04E:~/nutrialianza-monitoreo$ mysqlslap \
> --host=127.0.0.1 \
> --port=3306 \
> --protocol=tcp \
> --user="$MYSQL_USER" \
> --password="$MYSQL_PASSWORD" \
> --create-schema="$MYSQL_DATABASE" \
> --concurrency=220 \
> --iterations=1 \
--query=> --query="SELECT SLEEP(10)" \
> --no-drop
mysqlslap: [Warning] Using a password on the command line interface can be insecure.
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections

```

El resultado confirmó de forma directa la condición esperada. MySQL comenzó a rechazar conexiones con el código 1040 “Too many connections”. El benchmark reportó un tiempo de 16.492 segundos para completar la ejecución, 220 clientes y un promedio de 1 consulta por cliente.

```
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
mysqlslap: Error when connecting to server: 1040 Too many connections
Benchmark
  Average number of seconds to run all queries: 16.492 seconds
  Minimum number of seconds to run all queries: 16.492 seconds
  Maximum number of seconds to run all queries: 16.492 seconds
  Number of clients running queries: 220
  Average number of queries per client: 1
```

6. Verificación del estado de MySQL

Después de finalizar mysqlslap se consultaron los contadores globales del servidor.

Connection_errors_max_connections pasó de 0 al inicio de la preparación a 897 después de la prueba. Este valor representa intentos de conexión rechazados por haber alcanzado el límite de conexiones; no equivale a 897 usuarios simultáneos, ya que el contador acumula los intentos fallidos registrados por MySQL.

En la misma verificación, Threads_connected devolvió 1. Esto demuestra que las conexiones temporales utilizadas durante el benchmark se liberaron y que el servicio recuperó un nivel normal de sesiones activas después de la saturación.

```
julio@DESKTOP-JN2R04E:~/nutrialianza-monitoreo$ mysql -h 127.0.0.1 -P 3306 \
> -uroot \
$MYS> -p"$MYSQL_ROOT_PASSWORD" \
> -e "SHOW GLOBAL STATUS LIKE 'Connection_errors_max_connections'; SHOW GLOBAL STATUS LIKE 'Threads_connected'"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Connection_errors_max_connections | 897 |
+-----+-----+
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Threads_connected | 1 |
+-----+-----+
```

7. Detección en Prometheus

Durante la prueba, Prometheus recibió del mysqld-exporter el incremento del contador asociado a max_connections. La consulta de N8N evaluó la diferencia producida dentro de una ventana de un minuto. Esto permitió activar la alerta únicamente cuando hubo rechazos recientes y evitó que el contador acumulado mantuviera una alarma permanente.

En una captura posterior, la consulta increase(...[1m]) puede aparecer nuevamente en 0. Ese resultado es correcto si transcurrió más de un minuto desde el último rechazo: la métrica acumulada continúa en 897, pero ya no existen errores nuevos dentro de la ventana temporal consultada. La llegada de la alerta a Telegram confirma que el valor fue mayor que 0 durante la ventana del incidente y que N8N procesó esa condición.

```
julio@DESKTOP-JN2R04E:~/nutrialianza-monitoreo$ curl -sG http://localhost:9090/api/v1/query \
--data-urlencode 'query=increase(mysql_global_status_connection_errors_total{error="ma> --data-urlencode 'query=in
crease(mysql_global_status_connection_errors_total{error="max_connections"}[1m])'
{"status":"success","data":{"resultType":"vector","result":[{"metric":{"error":"max_connections","instance":"mysqld-e
xporter:9104"},"job":"mysql"},"value":[1786775832.223,"0"]}]}"}j
ulio@DESKTOP-JN2R04E:~/nutrialianza-monitoreo$
```

8. Automatización con N8N, Groq y Telegram

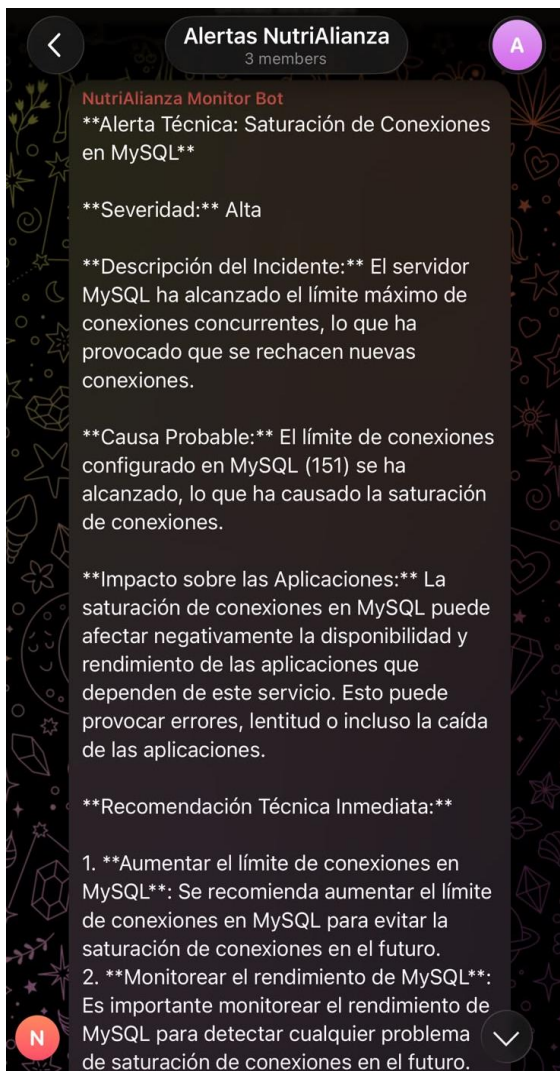
El workflow del Escenario 2 quedó separado del flujo utilizado para Nginx. Cuando la consulta de Prometheus detectó rechazos recientes, el nodo JavaScript construyó un prompt con el tipo de incidente, el límite configurado de 151 conexiones y la cantidad detectada. Groq generó una alerta en español con severidad, descripción, causa probable, impacto y recomendaciones técnicas.

Telegram recibió automáticamente el mensaje del bot “NutriAlianza Monitor Bot”. La alerta clasificó el evento con severidad alta, identificó que MySQL había alcanzado el límite máximo de conexiones concurrentes y señaló como impacto posible la pérdida de disponibilidad, lentitud y errores en las aplicaciones dependientes de la base de datos.

```

40  return [
41    {
42      json: {
43        tipoIncidente: "Saturación de conexiones MySQL",
44        rejectedConnections,
45        maxConnections: 151,
46        prompt
47      }
48    }
49  ];

```



9. Interpretación técnica de los resultados

- **La saturación fue real y medible.** El error 1040 apareció durante `mysqlslap` y el contador `Connection_errors_max_connections` alcanzó 897.
- **La carga superó la capacidad configurada.** Se solicitaron 220 clientes concurrentes frente a un máximo de 151, una diferencia de 69 conexiones potenciales por encima del límite.
- **El servicio se recuperó al finalizar.** `Threads_connected` descendió a 1 después de liberar las conexiones del benchmark.
- **La detección evitó alertas permanentes.** La consulta `increase(...[1m])` se enfoca en rechazos nuevos y vuelve a 0 cuando el incidente queda fuera de la ventana de un minuto.
- **La automatización funcionó de extremo a extremo.** Prometheus entregó la señal, N8N evaluó la condición, Groq redactó el diagnóstico y Telegram recibió la notificación.

10. Conclusión

El Escenario 2 cumplió el objetivo de emular una saturación de conexiones en MySQL y comprobar la respuesta del sistema de monitoreo. La prueba generó rechazos reales con el error 1040, elevó el contador `Connection_errors_max_connections` a 897 y dejó evidencia de recuperación del servicio con `Threads_connected = 1` al finalizar.

La observabilidad también funcionó como se esperaba. `mysqld-exporter` expuso las métricas, Prometheus las almacenó, N8N detectó los rechazos dentro de la ventana temporal configurada, Groq produjo el análisis técnico y Telegram entregó la alerta al equipo de TI. El resultado demuestra que la solución no se limita a recolectar métricas: también correlaciona un evento operativo con una notificación comprensible y accionable.

11. Guía de evidencias para la entrega

Para documentar el escenario con trazabilidad visual, se recomienda colocar las siguientes capturas en los espacios reservados del documento:

- Captura de `mysql_up = 1` y métricas base antes de la saturación.
- Captura de `mysqlslap` con los errores `1040 Too many connections`.
- Captura del benchmark con 220 clientes y 16.492 segundos.
- Captura de `Connection_errors_max_connections = 897` y `Threads_connected = 1`.
- Captura de Prometheus/N8N asociada al incidente.
- Captura de la alerta recibida en Telegram.

Referencia de la actividad: Guía del proyecto NutriAlianza S.A., Tarea 4 - Emulación de Saturación en Producción, escenario obligatorio de saturación de conexiones MySQL mediante `mysqlslap`.